Student Marks Management API

An ASP.NET Core service with PostgreSQL persistence, JWT role rules, graded reports and repeatable API testing.

Asif Baig | Registration: **23F-0053**
FAST NUCES, Chiniot-Faisalabad Campus | 10 November 2026

Repository
https://github.com/asifbaig7034/23F-0053_ASIF_Assignment1

Architecture and responsibilities

Endpoints	Services	EF Core	PostgreSQL
HTTP, DTO validation and access policies	Interfaces and application operations	Scoped DbContext and migrations	Tables, indexes and constraints

JWT authentication precedes protected endpoints. Teacher policies guard writes and administration. Student record/transcript access checks the linked **studentId** claim.

Runtime and tooling

.NET 10 / ASP.NET Core
PostgreSQL 17.11
EF Core + Npgsql
Swagger UI and Postman

Recorded implementation results

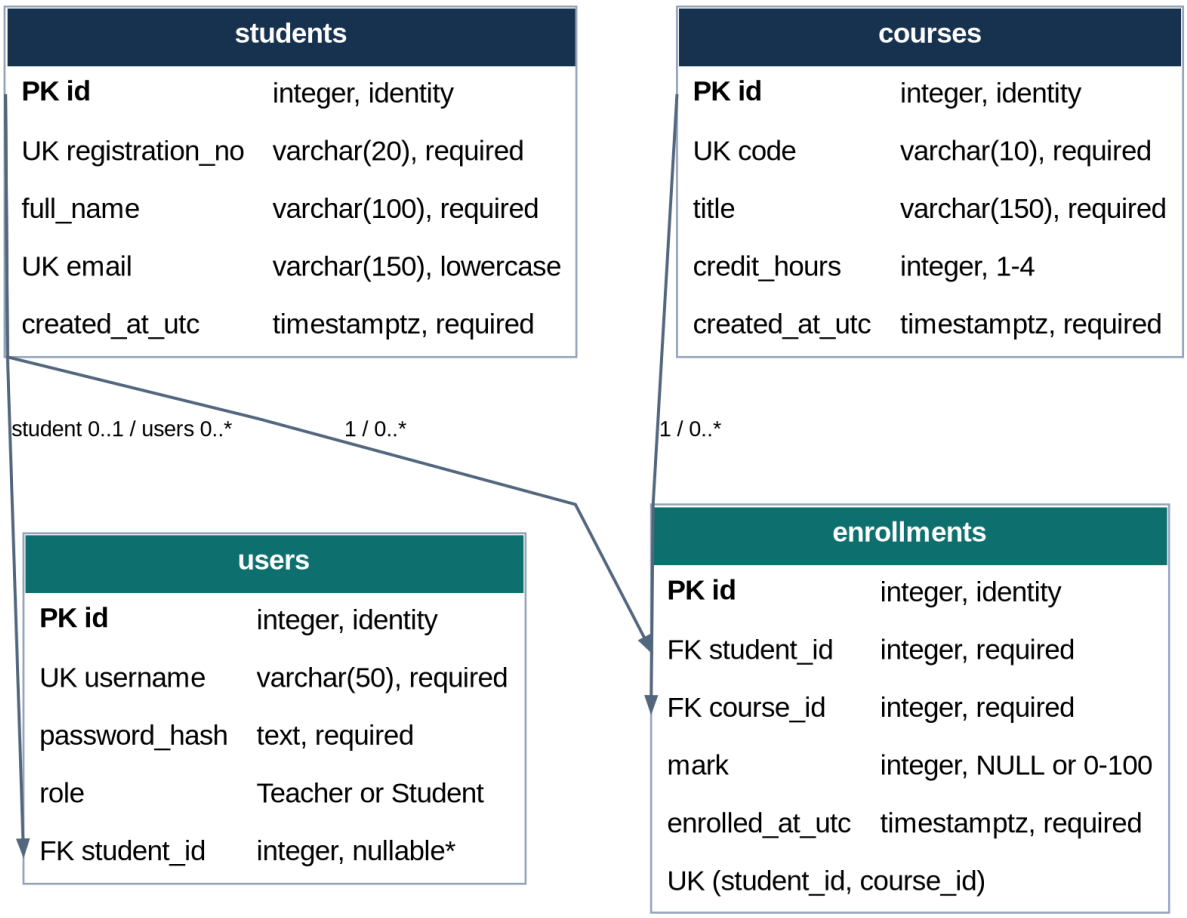
118 / 118 Postman tests passed
107 / 107 smoke checks passed
Smoke process exit code: **0**
HTTPS redirect and CORS passed

Production practices

CamelCase DTOs, UTC timestamps and Problem Details form the API contract. Read-only queries use AsNoTracking and projections; database calls use async methods and cancellation tokens. ILogger records operations without logging passwords or JWTs.

Final verification pending: run scripts/Finish-Assignment.ps1 to verify a fresh database and clean Git clone, capture live evidence and regenerate the final PDF.

Schema and integrity



PK = primary key; FK = foreign key; UK = unique index. EF infrastructure table __EFMigrationsHistory is omitted. Relationships reflect the actual exported schema.

Database guarantees

Unique indexes protect registration numbers, lowercase emails, course codes, usernames and student/course enrollment pairs. Credit hours are 1-4; marks are NULL or integer 0-100. All three foreign keys use ON DELETE RESTRICT.

Account link and deletion rules

A Student-role user must have a student link. The link is nullable for other roles; its FK index is non-unique, so the database permits multiple accounts per student. Deleting an enrolled student/course or a student with a linked account returns 409.

Concurrency: PostgreSQL unique indexes decide duplicate inserts atomically. SQLSTATE 23505 maps to 409. Migrations initialize the application; schema.sql is the actual schema-only export.

Authentication and configuration

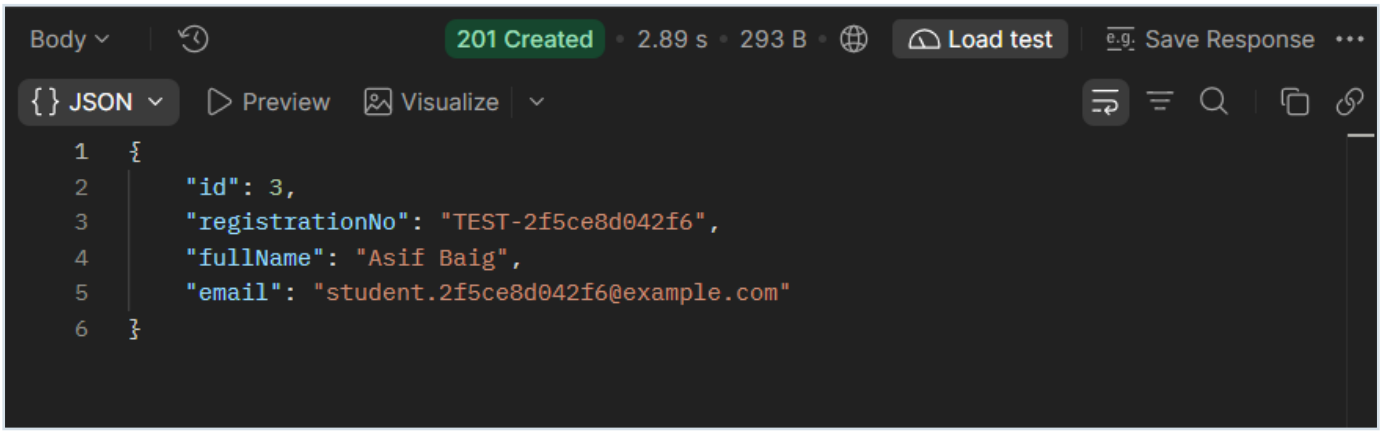
Passwords use ASP.NET Core PasswordHasher. Signed JWTs include expiry and role claims. User-secrets hold development credentials/signing keys; environment variables support deployment. CORS permits configured origins. HTTP redirects to HTTPS; Swagger UI is available in Development.

Grades and input validation

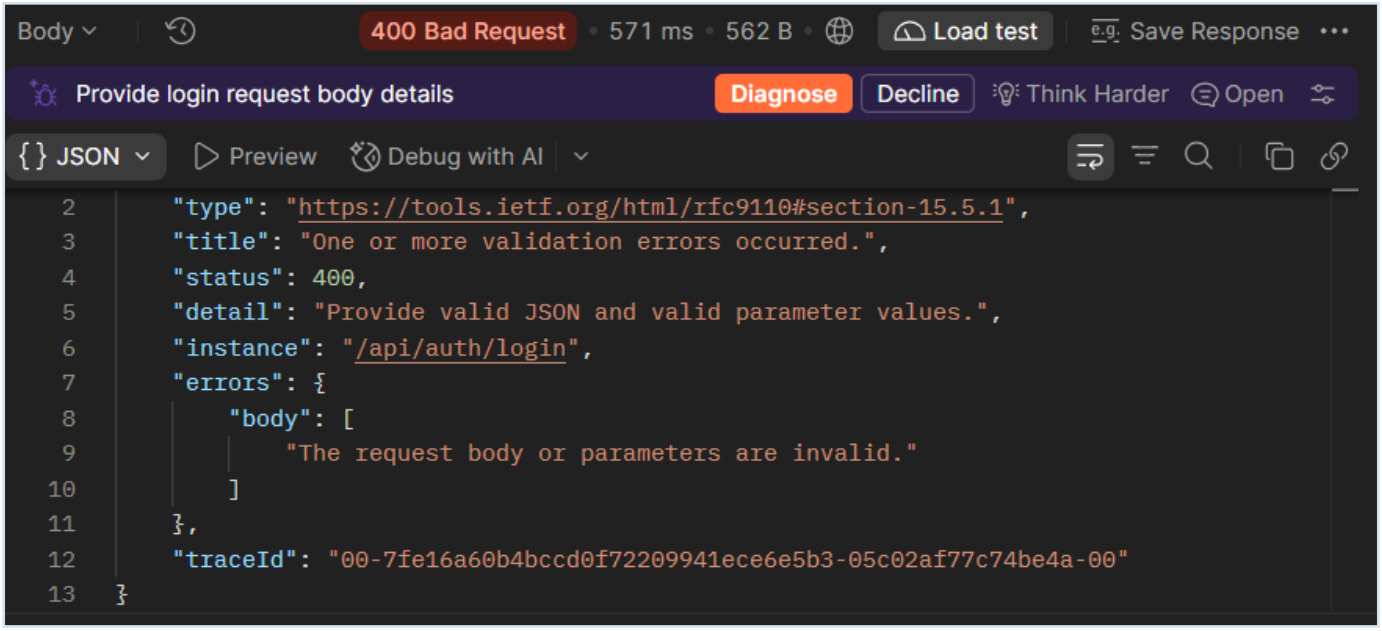
Weighted average: sum(mark x credits) / sum(credits), using graded courses only.
Seed example: (80 x 3 + 60 x 2) / (3 + 2) = **72**. CS401 average: (80+70)/2 = **75**.

NULL means ungraded and is excluded from averages. Zero is graded. Empty/ungraded averages are NULL. Passing uses Grading:PassMark (50).

201 Created | POST /api/students - successful DTO response



400 Bad Request | POST /api/auth/login - missing/invalid request body

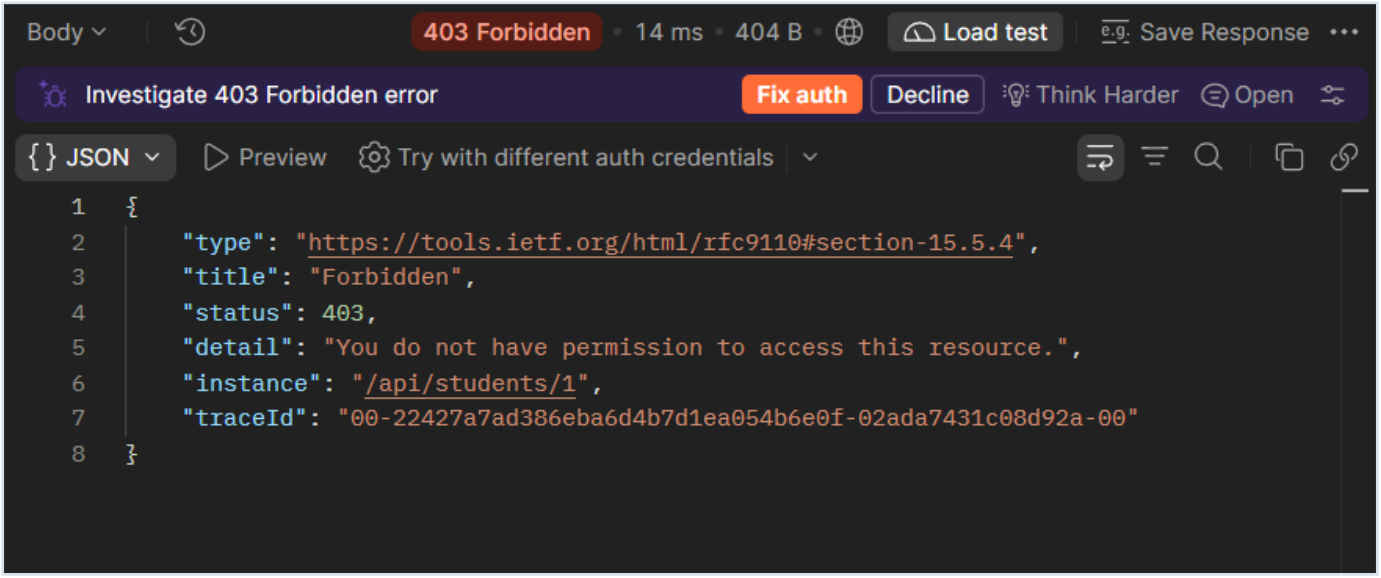


Captured during implementation. Original complete screenshots are included in docs/screenshots; the relevant response areas are shown here.

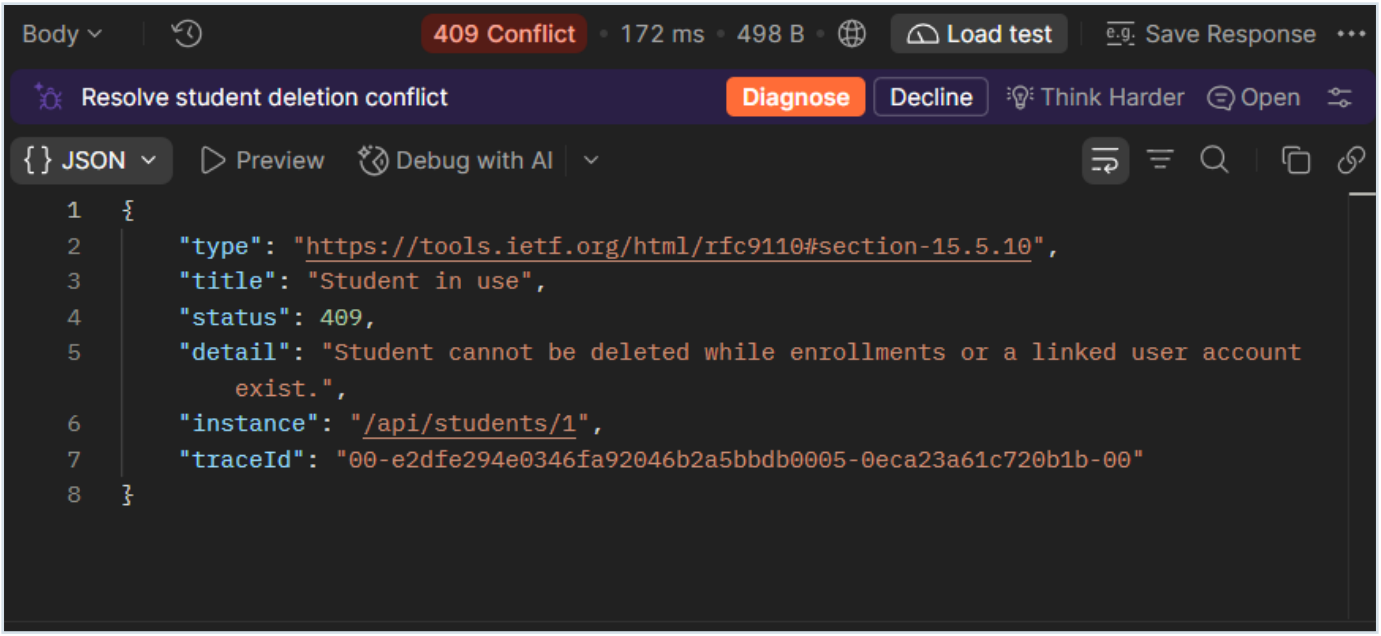
Expected error behavior

401 requires valid authentication; 403 means the caller lacks permission. 404 identifies a missing resource. 409 reports a conflict with existing data. Error bodies contain status, title and detail.

403 Forbidden | Student attempts DELETE /api/students/1



409 Conflict | Teacher attempts to delete an enrolled student



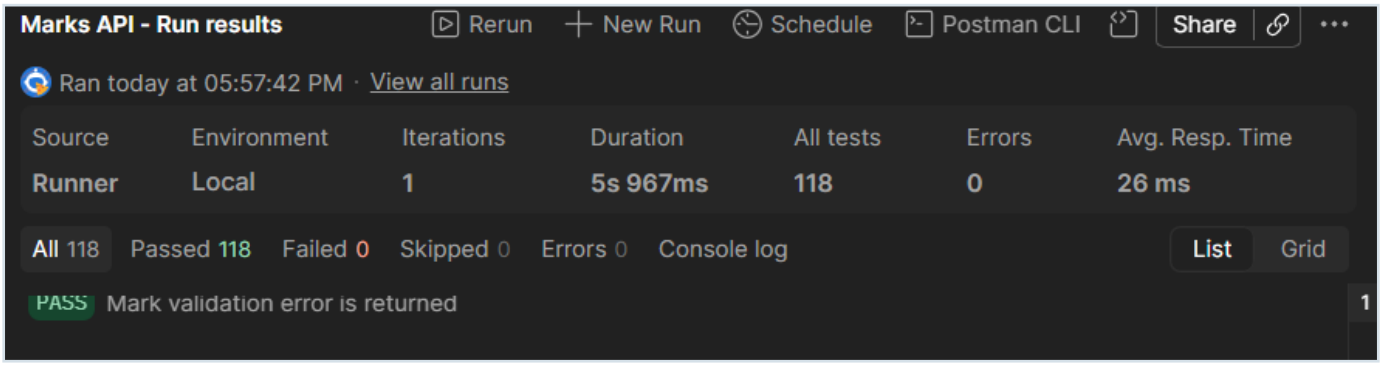
Issues resolved during implementation

PostgreSQL tools were located through their installation bin path. Credentials were configured through user-secrets. Postman login scripts kept Teacher/Student tokens separate; outdated URLs, methods and request ordering were corrected before the passing full run.

Repeatable verification

The collection has **59 requests, 118 tests and 32 negative cases**. Auth runs first. Generated IDs chain through CRUD; cleanup precedes the final negative tests. Exported token values are blank.

Postman Collection Runner | 118 passed, 0 failed, 0 errors



Startup smoke module

Opt-in --smoke-test starts the real HTTP server, uses PostgreSQL, checks success/error cases and report edge cases, deletes SMOKE-prefixed fixtures and exits 0 on success. Credentials come from configuration; normal startup does not run tests.

```
Terminal result supplied during implementation:
SMOKE SUMMARY: 107 passed, 0 failed
Second exit code: 0

The final helper inserts a real fresh-run screenshot.
```

Pending local final check: run Finish-Assignment.ps1. It verifies a clean Git clone and a separate fresh database, captures actual smoke/Postman windows and regenerates this report.

Final results, source commit, database name, seed/cleanup counts and two smoke runs are recorded in docs/verification-results.json. Existing application data is retained. The screenshot helper uses actual on-screen windows.

Primary references

ASP.NET Core JWT authentication
EF Core migrations
PostgreSQL constraints
Postman Collection Runner